



Penetration Testing

Propto (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Penetration Test Scope	7
Security Rating	8
Severity Definitions	9
Status Definitions	10
Penetration Test Findings	11
XRPL Security and Robustness Evaluation	26
Conclusion	30
Our Methodology	31
Disclaimers	33
About Hashlock	34

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Reclosure team partnered with Hashlock to conduct a security Penetration Test of their Propto Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Propto is a next-generation digital asset platform that empowers users to navigate and optimize their exposure to decentralized finance. It provides advanced trading, investment, and analytics tools within an intuitive interface, enabling both novice and experienced investors to manage and grow their crypto portfolios with confidence. The platform leverages cutting-edge blockchain technology to ensure transparency, security, and efficiency in every transaction, helping users stay ahead in the rapidly evolving digital asset market.

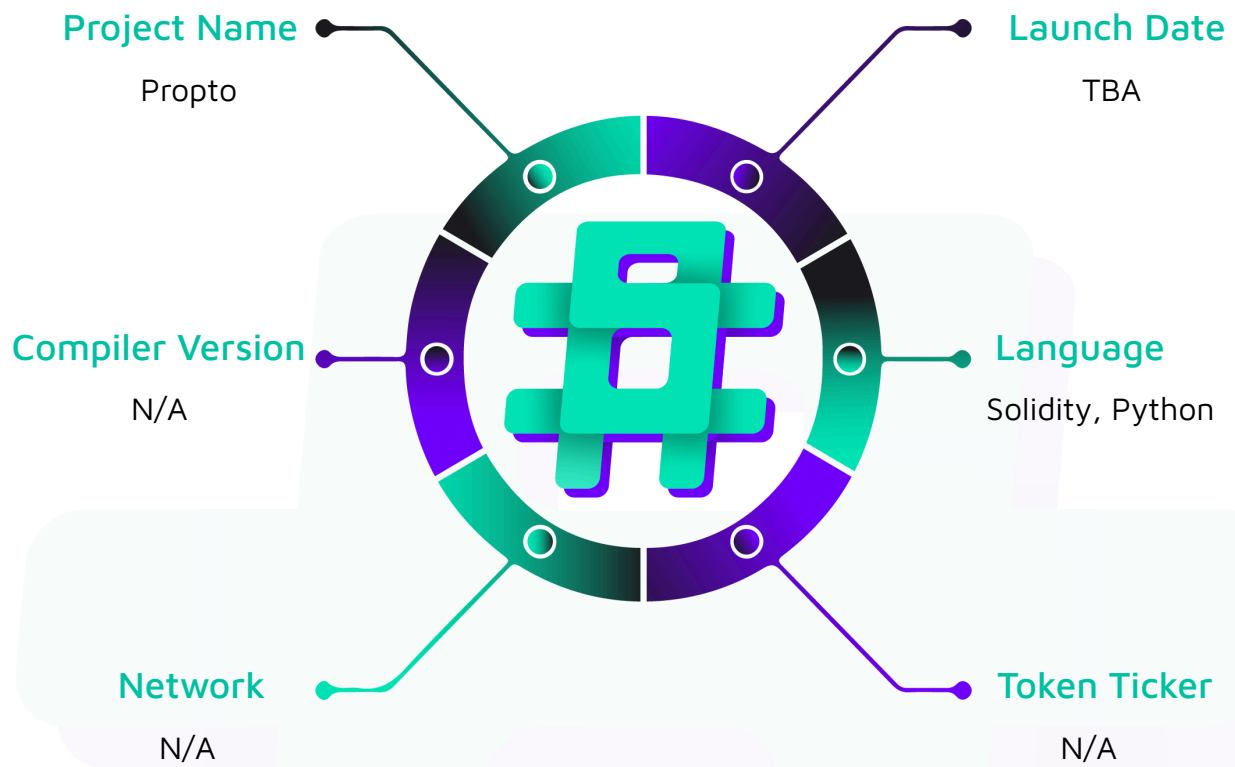
Project Name: Propto

Project Type: Defi

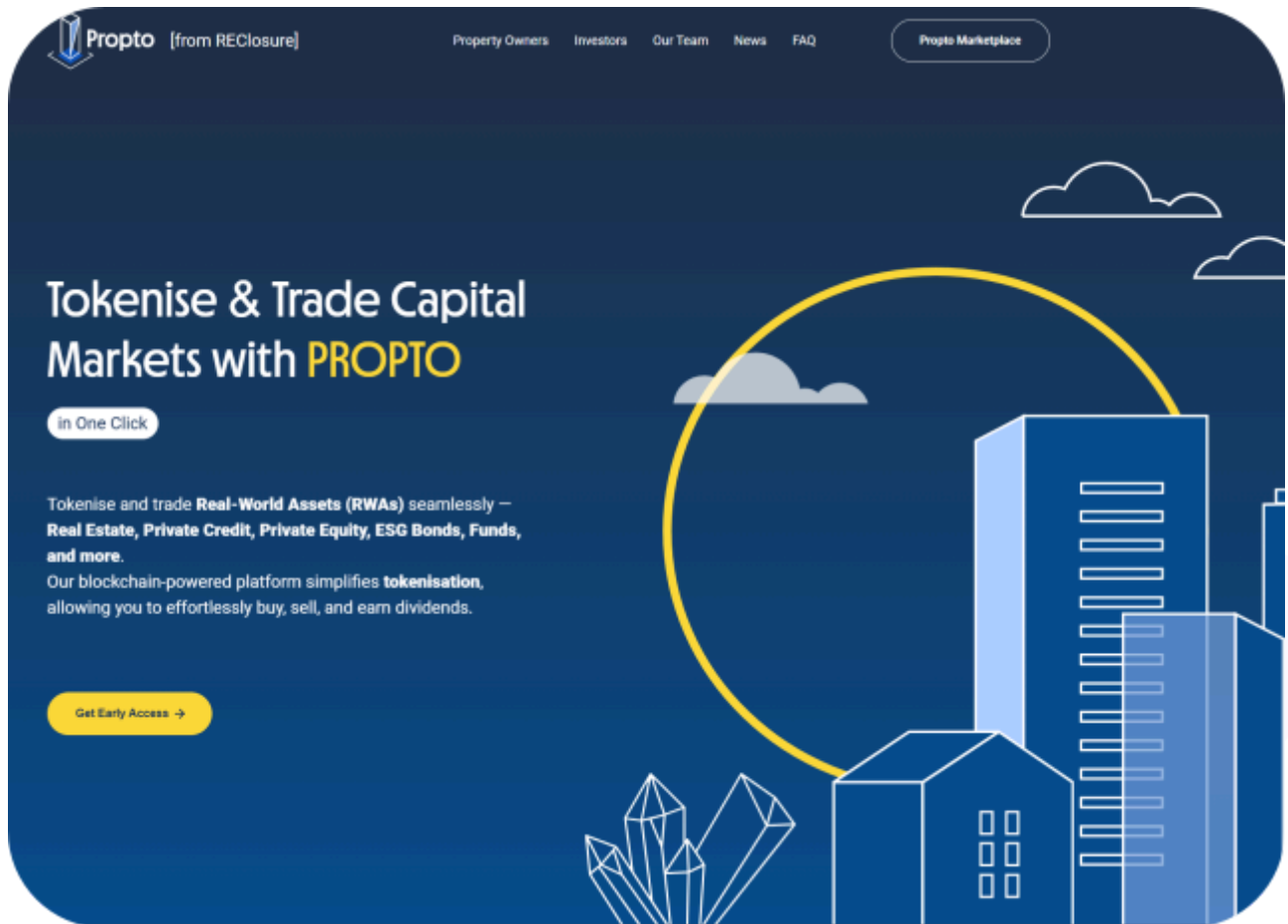
Website: <https://www.propto.com/>

Logo:



Visualised Context:

Project Visuals:



Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Propto project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	Propto Project code base
Platform	Web apps & APIs
Penetration Test Date	March, 2025
Repository/Website URL/IP Tested	https://github.com/reclosure/propto-backend
GitHub Commit Hash	cf5177a608237ebac7273dfdf7a9c1ee0ffc0caa

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Penetration Test Findings](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High-severity vulnerabilities

4 Medium-severity vulnerabilities

4 Low-severity vulnerabilities

4 QAs

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] `app/controller/accesscontrol/accesscontrol.py` - Vulnerabilities in Role-Based Access Control implementation

Description

The access control subsystem contains fundamental design flaws enabling privilege escalation and unauthorized financial operations.

Vulnerability Details

The `check_policy()` function returns `True` by default for 18+ high-risk actions including `CLAWBACK_FROM_USER` and `SETUP_USER_TRUSTLINES`, effectively disabling security controls. A critical typographical error ("`Admin`" vs "`ADMIN`") creates inconsistent role validation while missing ownership checks permit users to execute privileged transactions without proper authorization context.

Impact

Attackers could illegally recover funds via clawback mechanisms, manipulate asset trustlines, and bypass financial freezing controls. These flaws enable both horizontal and vertical privilege escalation, potentially resulting in substantial financial losses.

Recommendation

Restructure the authorization framework using a deny-by-default approach with explicit permission grants. Implement transaction signing requirements for sensitive financial operations. Replace string-based role checks with enumerated types to prevent typographical errors.

Status

Resolved

[H-02] compliance/ - Path traversal vulnerability

Description

The file upload functionality contains critical path sanitization flaws enabling unauthorized file system access. In concerns both `propto-backend-audit/app/controller/compliance/amlbotkyc.py` and `propto-backend-audit/app/controller/compliance/compliance.py`

Vulnerability Details

The `upload_file()` method accepts user-supplied `file_path` values without proper validation or normalization. Attackers can craft malicious paths containing directory traversal sequences (e.g., `../../etc/passwd`) to access arbitrary system locations. The current implementation relies solely on `os.path.exists()` without verifying whether the path resides within an allowed directory tree.

Impact

Authenticated adversaries could exploit this vulnerability to read sensitive system files, overwrite application binaries, or establish persistent access through planted executable files. This breach could lead to a complete system compromise.

Recommendation

Implement strict path validation using `os.path.realpath()` combined with a whitelist of permitted directories. Generate randomized filenames for stored uploads rather than preserving original names. Enforce file extension verification against approved types (e.g., `.jpg`, `.png`).

Status

Resolved

[H-03] app/controller/auth/auth.py - JWT authentication algorithm confusion vulnerability in OLD validation function

Description

The authentication module contains a vulnerable JWT validation implementation susceptible to algorithm confusion attacks. When Auth0 validation fails, the system falls back to a legacy validation method that inadequately verifies token algorithms, potentially allowing attackers to forge valid authentication tokens using the "alg: none" attack.

Vulnerability Details

The `_OLD_validate_access_token` function is used as a fallback authentication:

```
async def _OLD_validate_access_token(token, access_type):

    token_secret_key = TOKENS[access_type]['secret_key']

    token_expire_minutes = TOKENS[access_type]['expire_minutes']

    token_algorithm = TOKENS[access_type]['algorithm']

    try:

        payload = jwt.decode(token, token_secret_key, algorithms=[token_algorithm])

    except jwt.exceptions.ExpiredSignatureError:

        raise InvalidCredentialsException("Expired token")

    except:

        raise InvalidCredentialsException("Invalid credentials")

    # jwt.decode should also check for expiry, but let's be safe

    expiry_time = payload.get('exp')
```

```

if time.time() > expiry_time:

    raise InvalidCredentialsException("Expired token")

email = payload.get("sub")

return email

```

This function is called from `_validate_access_token` when Auth0 validation fails:

```

async def _validate_access_token(token, access_type):

    #TODO Uncomment this assert when removing

    # native authentication

    #assert access_type=="login", "Only login supported"

    result = auth0.VerifyToken(token=token).verify()

    if not "user_info" in result:

        #TODO This is a temporary solution before we remove//@audit-info todo

        # native authentication

        return await _OLD_validate_access_token(token, access_type)

    return result['user_info']['email']

```

Impact

An attacker could craft a token with the "none" algorithm to bypass signature verification entirely, gaining unauthorized access to protected resources, including administrative endpoints. This could lead to privilege escalation, data exposure, and complete system compromise.

Recommendation

Remove the legacy authentication path immediately and rely solely on Auth0.

Status

Resolved

Medium

[M-01] **app/controller/accesscontrol/accesscontrol.py** - **Hardcoded superadmin authentication token**

Description

The `is_superuser()` function in the access control module uses a static, environment-based token for superadmin authentication, introducing a significant security vulnerability. This implementation allows unrestricted system access through a single, potentially long-lived credential that cannot be dynamically managed or revoked. The hardcoded superadmin token represents a significant security risk, as it creates a persistent backdoor into the system. If the token is compromised through any means—such as source code exposure, environment file leaks, or insider threats—an attacker gains complete system access without any additional authentication barriers.

Vulnerability Details

The `is_superuser` function:

```
def is_superuser(token):  
    return token==get_env_var("SUPERADMIN_TOKEN")
```

It checks if the token is used. There is no role-based approach.

Impact

An attacker who obtains the superadmin token gains unrestricted access to all system functionalities. The token cannot be easily managed or monitored.

Recommendation

Replace the static superadmin token with a robust authentication mechanism, for example, implement the Superadmin role as part of the RBAC.

Note

The Propto team informed Hashlock that the token is ephemeral and not hardcoded in source or environment. Which means it is tied to single-session and cannot be reused.

Status

Resolved

[M-02] app/controller/auth/auth0.py - Lack of HTTP request timeout configuration

Description

The authentication module's HTTP client configuration in app/controller/auth/auth0.py lacks a timeout mechanism for external API calls. Without explicit timeout settings, the application remains vulnerable to potential denial of service (DoS) scenarios and unexpected connection hanging during network interactions.

Uncontrolled HTTP requests can block application threads indefinitely, potentially causing system unresponsiveness when external services are slow or unreachable. This vulnerability creates a risk of resource exhaustion and reduced application reliability.

Vulnerability Details

The example code from app/controller/auth/auth0.py:

```
async with httpx.AsyncClient() as client:

    response = await client.get(user_info_url, headers=headers)

    if response.status_code == 200:

        return response.json()

    else:

        return {"status": "error", "message": "Failed to fetch user info",
"details": response.text}
```


It should be noted that `httpx` is used widely throughout the project, also without a required timeout.

Impact

Misconfigured or temporarily not working external services could consume application resources by delaying responses, potentially causing application-wide performance issues or complete service interruption.

Recommendation

Implement a 10-second timeout for all external HTTP requests using `httpx`'s timeout configuration.

Status

Resolved

[M-03] `app/controller/compliance/amlbotkyc.py` - Sensitive information exposure in logs

Description

The `AMLBotKYC` implementation exposes sensitive authentication credentials and API responses through insecure logging practices, violating confidentiality requirements.

Vulnerability Details

`upload_file()` logs full headers containing API keys (Token `{self.api_key}`)

`handle_errors()` logs complete error responses including potentially sensitive server data

Logging occurs at `INFO` level, making these exposures persistent in normal operation logs

Impact

Compromised logging channels could lead to API credential theft and exposure of PII/sensitive system information. Attackers could use leaked credentials to impersonate the application to external services.

Recommendation

Redact sensitive fields from logs using pattern masking. Implement sensitive data classifiers for log sanitation. Restrict API key logging to DEBUG level only.

Status

Resolved

[M-04] `app/controller/compliance/amlbotkyc.py` - Password policy is not enforced during password reset

Description

KYC processing components improperly log sensitive personal information violating data protection regulations.

Vulnerability Details

The `create_applicant()` method logs complete applicant data structures containing government IDs, birthdates, and residential information. The `handle_applicant_reviewed()` function records verification status changes without pseudonymization, exposing real names and verification outcomes.

```
def create_applicant(self, applicant_data):  
  
    """Create an applicant with the provided data."""  
  
    url = f"{self.base_url}/applicants"  
  
    headers = self.get_headers()  
  
    try:  
  
        response = requests.post(url, json=applicant_data, headers=headers)
```

```
response.raise_for_status()

return response.json()

except requests.exceptions.HTTPError as e:

    self.handle_errors(e.response)

    return None

except requests.exceptions.RequestException as e:

    log_info(f"Error creating applicant: {e}")

    return None
```

Impact

Exposed PII enables identity theft and fraudulent account takeovers. This could create a conflict with the regulation, as this way PII should not be processed.

Recommendation

Implement hashed user references using SHA-3-256 derived identifiers. Develop log redaction filters for 88+ PII patterns including dates, IDs, and geographic markers. Encrypt audit logs containing sensitive processing metadata.

Status

Resolved

Low

[L-01] `app/controller/cache.py` and `decorators.py` - Usage of Pickle is insecure

Description

The cache implementation in `app/controller/cache.py` and `/app/controller/decorators.py` uses Python's `pickle` module for data deserialization, introducing potential security vulnerabilities. Pickle deserialization can execute arbitrary code if the serialized data is compromised, creating a significant risk for remote code execution.

While the current implementation uses internal database storage, this approach poses substantial security risks during potential future integrations or if the cache mechanism is extended. Pickle lacks built-in security mechanisms and can unpickle maliciously crafted objects that may execute unauthorized code.

An attacker with the ability to manipulate cached data could potentially inject malicious code that would be executed during deserialization, leading to unauthorized system access, code execution, or other critical security breaches.

Recommendation

Replace pickle with safer serialization methods like JSON or MessagePack.

Status

Resolved

[L-02] `app/controller/comms/email.py` - Insufficient validation in `sendmail` function

Description

The `sendmail` function in `app/controller/comms/email.py` demonstrates inadequate input validation for email communication parameters.

The function partially sanitizes the subject line by replacing newline characters (`\n`), but does not account for full newline sequences such as `\r\n`.

Due to this, an attacker could potentially manipulate email parameters to add arbitrary email headers or content, leading to email spoofing.

Recommendation

Validate also `\r\n` sequence in addition to `\n` alone.

Status

Resolved

[L-03] `app/controller/main.py` - Missing Admin role verification on administrative endpoints

Description

The application has administrative endpoints that only validate user authentication (token validity) without verifying whether the authenticated user has admin privileges. This represents an authorization bypass vulnerability where any user with a valid token can access administrative functions.

Vulnerability Details

The administrative endpoint only verifies token validity but doesn't check admin status:

```
@app.get("/admin/forms/{form_type}", response_model=List[outputs.Form],
status_code=200, tags=["Admin"])

async def get_company_form(form_type: common.FormType, user: str =
Depends(validate_token)):

    """

    Get forms for approval

    """

    res = await controller.get_forms_by_type(form_type, user)
```

```
return res
```

The function gets the user name but does not check if it's admin:

```
async def validate_token(token: str = Depends(oauth2_scheme)):  
    user = await controller.get_user_from_token(token)  
    return user
```

The `validate_token` dependency only confirms authentication, but not the authorization level.

Impact

Any authenticated user, regardless of their role, can access administrative endpoints and perform privileged actions like approving forms.

Recommendation

Implement proper role-based access control (RBAC). Verify admin status before allowing access to admin endpoints.

Status

Acknowledged

[L-04] `app/controller/main.py` - Potential Denial of Service via Invitations

Description

The `get_invitations_for_user()` function in `app/controller/main.py` lacks protection against potential resource exhaustion through excessive invitation generation and retrieval.

A malicious user could potentially create a large number of invitations and repeatedly call this endpoint, potentially overwhelming database resources and degrading system performance. This is because `get_invitations_for_user` iterates over all user invitations each time it's called.

Recommendation

Implement pagination, rate limiting, and/or maximum invitation retrieval constraints to mitigate potential abuse.

Status

Resolved

QA

[Q-01] `app/scratch/main.py` - Insufficient upload validation

Description

The test suite revealed multiple instances of inadequate file upload validation, potentially exposing the application to security risks associated with unrestricted file uploads. However, since this is a Test Suite, it is rated as QA.

Multiple functions in the TestSuite class demonstrate poor file upload security practices:

1. `upload_file()` method lacks comprehensive input validation:
 - No file type checking
 - No file size restrictions
 - Minimal parameter validation
 - Accepts arbitrary file content without sanitization
2. `put_presigned_upload()` method:
 - Bypasses application-level controls
 - Allows direct file uploads via presigned URLs
 - No additional security checks on uploaded content
3. `get_random_file()` function generates files with minimal content validation, potentially facilitating testing of upload mechanisms without robust security controls

Recommendation

If the code will be used in production in the future:



- Validate file types
- Enforce strict file size limits
- Sanitize file names and content
- Implement content type checking
- Add server-side file type verification
- Implement shorter presigned URL timespan

Status

Resolved

[Q-02] `app/controller/database/database.py` - Lack of error handling

Description

In the database commit process, exceptions are silently suppressed using a bare `except` clause without logging or proper error handling. At line 133, any exception during `flag_modified(db_user, "data")` is caught and ignored with a `pass` statement, potentially masking critical database operation failures.

Recommendation

Catch and log the exception for better debugging capabilities instead.

Status

Resolved

[Q-03] `app/controller/database/database.py` - Usage of UUIDv1

Description

The application in line 243 uses `uuid.uuid1()` for generating document unique identifiers, which has weak randomness and predictability of the generated UUIDs. This is because UUID version 1 generates identifiers based on timestamp and MAC address, potentially making them more predictable and less secure for cryptographic purposes.

Recommendation

Replace `uuid.uuid1()` with `uuid.uuid4()`.

Status

Resolved

[Q-04] Multiple files - Unfinished developments**Description**

Multiple instances of commented-out code, TODO markers, and incomplete implementation were identified throughout the application, indicating potential technical debt and code quality issues. Identified code quality concerns include commented-out code segments suggesting incomplete feature implementation and unresolved TODO comments indicating pending development tasks.

Recommendation

Remove or complete commented-out code segments.

Status

Resolved

XRPL Security and Robustness Evaluation

XRPL Trading Audit Scope – Review and Confirmation of the XRPL DEX/trustline/trading implementation.

All components related to the XRPL DEX, trustline management, and trading implementation included in the referenced codebase were reviewed as part of the audit scope. This encompassed contract logic for trustline creation and deletion, order placement and execution mechanisms, as well as the interaction patterns with the XRPL decentralized exchange.

Following the identification and subsequent remediation of the vulnerabilities outlined in our findings, the current implementation demonstrates a significantly improved security posture. At the time of this assessment, no critical or high-risk issues remain unresolved, and the system is considered secure and robust for deployment under the reviewed scope. Continued adherence to best practices and periodic security reviews are still recommended as the protocol evolves.

DDOS Resilience on XRPL trading endpoints (rate limits, node fallback, retry strategies).

While the current codebase demonstrates baseline functionality for XRPL trading endpoints, additional infrastructure-level protections are recommended to ensure resilience against Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks. Specifically, we advise placing the application behind a Cloudflare Web Application Firewall (WAF) post-deployment to mitigate external threat vectors such as volumetric attacks, automated abuse, or endpoint flooding.

SQL Injection Mitigation – Validation of the ORM and Pydantic boundaries.

The application appears to utilize its Object-Relational Mapping (ORM) layer correctly, which inherently provides a degree of protection against SQL injection by abstracting raw query construction. Throughout our review, no instances of unsanitized or dynamically constructed SQL queries were identified.

Robustness of AUX Token Trading – Checking the trustline integrity, decimal math precision, and invariant enforcement.

No mathematical inaccuracies or precision-related issues were identified during our review of the AUX token trading logic. Decimal arithmetic appears to be handled correctly, and all token operations respect the precision constraints defined by the token standard.

Overall, the implementation demonstrates strong adherence to robustness principles in the context of AUX token trading.

Unsanitized User Data Handling – Confirming user input to be validated or sanitized before critical operations (esp. in compliance/KYC modules).

All previously missing or insufficient input validations—particularly within the compliance and KYC modules—have been identified and remediated. User input is now consistently validated or sanitized prior to any critical operations, effectively mitigating risks associated with malformed or malicious data.

With these fixes in place, the application enforces strict data integrity and adheres to security best practices when handling user-supplied information, especially in sensitive areas involving identity verification and regulatory compliance.

XRPL Node Connection DoS Vectors – Guarantee node timeouts, retry caps, and circuit breakers are in place.

The connection timeout issue related to XRPL node communication was identified during the review and has since been addressed. Proper timeout configurations are now in place, along with safeguards such as retry limits and fallback logic to prevent excessive load on a single node.

These improvements help mitigate potential Denial-of-Service (DoS) vectors stemming from unresponsive or slow XRPL nodes, contributing to greater overall stability and resilience of the application under adverse network conditions.

RBAC Coverage – Confirm edge cases (e.g., delayed role assignment, downgraded roles) are blocked.

All previously identified issues related to Role-Based Access Control (RBAC)—including edge cases such as delayed role assignment and improper handling of downgraded roles—have been addressed and resolved.

The current implementation enforces strict access controls, ensuring that permissions are accurately granted and revoked in real-time. All known edge cases are now properly handled, preventing unauthorized access or privilege escalation across the system. This reinforces the integrity of the access control model and aligns with best practices for secure role management.

Edge Cases in Payments/Trading – Check malformed amounts, trustline race conditions, and overflow conditions are handled.

All identified issues related to edge cases in payments and trading—such as malformed amounts, trustline race conditions, and overflow conditions—have been thoroughly addressed and remediated.

The current implementation ensures that malformed input is properly validated, preventing incorrect or malicious data from being processed. Additionally, race conditions related to trustline management have been resolved, and overflow

vulnerabilities have been safeguarded against through appropriate checks and controls. These fixes enhance the overall reliability and security of the payment and trading processes, ensuring smooth operation under various conditions.

Referral/Dividend flow – Check referral flow to make sure the payments are correctly processed for referrals/dividends

All identified issues related to the referral and dividend flow—ensuring that payments are correctly processed for referrals and dividends—have been thoroughly addressed and resolved.

The referral and dividend payment logic has been corrected, ensuring that rewards are accurately distributed based on the intended flow. All edge cases, such as incorrect payment calculations or improper handling of referral links, have been fixed, enhancing the integrity and reliability of the system.

Conclusion

After Hashlock's analysis, the Propto project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.